

Layer Normalization.

Before every sublayer, the transformer re-centres and re-scales each token's feature vector to zero mean and unit variance — *per token*, never across the batch. That single design choice is what lets a transformer handle sequences of any length. Below we derive the formula, normalize a 4-vector entirely by hand, and contrast it with BatchNorm — every number computed in full.

THE EQUATION

$$\text{LN}(x) = \gamma \odot \frac{x - \mu}{\sqrt{\sigma^2 + \varepsilon}} + \beta$$

Worked example. Normalize $x = [2.0, -1.0, 0.5, 1.5]$ ($d = 4$): $\mu = 0.75$, $\sigma^2 = 1.3125$, giving $\hat{x} \approx [1.0911, -1.5275, -0.2182, 0.6547]$ with $\gamma = \mathbf{1}$, $\beta = \mathbf{0}$.

Topic 2 of 5 in this module · companion to the course page · v1.0

Contents

1	The equation	2
2	Worked computation, fully by hand	3
3	The role of ε	4
4	Contrast with BatchNorm	4
5	Where this sits in the track	5
A	Reproducible (NumPy)	5

1 The equation

Layer normalization acts on a single token's feature vector $x \in \mathbb{R}^d$. It subtracts the feature-wise mean, divides by the feature-wise standard deviation, then applies a learned per-feature scale γ and shift β :

$$\text{LN}(x) = \gamma \odot \frac{x - \mu}{\sqrt{\sigma^2 + \varepsilon}} + \beta \quad (1)$$

where the statistics are taken *over the d feature dimensions of that one token*:

$$\mu = \frac{1}{d} \sum_{i=1}^d x_i, \quad \sigma^2 = \frac{1}{d} \sum_{i=1}^d (x_i - \mu)^2 \quad (2)$$

Symbol	Shape	Meaning
x	$\mathbb{R}^d$	one token's feature vector
μ	scalar	mean over the d features of <i>this</i> token
σ^2	scalar	variance over the d features of this token
γ	$\mathbb{R}^d$	learned per-feature scale (gain)
β	$\mathbb{R}^d$	learned per-feature shift (bias)
ε	scalar	small constant ($\sim 10^{-5}$) for numerical safety
$\odot$	—	elementwise (Hadamard) product

LN asks of each token, “relative to your own features, which are big and which are small?” It throws away the overall level (μ) and overall spread (σ) of that token and keeps only the *shape*. Because the question is answered using only that token’s own numbers, it does not matter how many other tokens are in the sequence or batch — one token, ten tokens, or a thousand, each is normalized identically and independently.

2 Worked computation, fully by hand

Take $x = [2.0, -1.0, 0.5, 1.5]$ with $d = 4$, and $\gamma = [1, 1, 1, 1]$, $\beta = [0, 0, 0, 0]$.

Step 1 — mean. Sum the features and divide by d :

$$\mu = \frac{2.0 + (-1.0) + 0.5 + 1.5}{4} = \frac{3.0}{4} = 0.75.$$

Step 2 — centre. Subtract μ from each entry:

$$x - \mu = \begin{bmatrix} 2.0 - 0.75 \\ -1.0 - 0.75 \\ 0.5 - 0.75 \\ 1.5 - 0.75 \end{bmatrix} = \begin{bmatrix} 1.25 \\ -1.75 \\ -0.25 \\ 0.75 \end{bmatrix}.$$

Step 3 — variance. Square each centred entry, average:

$$(x - \mu)^2 = \begin{bmatrix} 1.5625 \\ 3.0625 \\ 0.0625 \\ 0.5625 \end{bmatrix}, \quad \sigma^2 = \frac{1.5625 + 3.0625 + 0.0625 + 0.5625}{4} = \frac{5.25}{4} = 1.3125.$$

Step 4 — scale. With $\varepsilon = 10^{-5}$,

$$\sqrt{\sigma^2 + \varepsilon} = \sqrt{1.31251} = 1.1456483.$$

Divide the centred vector by this:

$$\hat{x} = \frac{x - \mu}{\sqrt{\sigma^2 + \varepsilon}} = \begin{bmatrix} 1.25/1.1456483 \\ -1.75/1.1456483 \\ -0.25/1.1456483 \\ 0.75/1.1456483 \end{bmatrix} = \begin{bmatrix} 1.09109 \\ -1.52752 \\ -0.21822 \\ 0.65465 \end{bmatrix}.$$

Step 5 — affine. With $\gamma = \mathbf{1}$, $\beta = \mathbf{0}$, $\text{LN}(x) = \gamma \odot \hat{x} + \beta = \hat{x}$.

Check. The output has mean ≈ 0 and variance ≈ 1 :

$$\frac{1}{4} \sum_i \hat{x}_i \approx -2.8 \times 10^{-17} \approx 0, \quad \frac{1}{4} \sum_i \hat{x}_i^2 \approx 0.99999 \approx 1.$$

The heatmap shows the magnitude of each normalized feature (intensity scales with $|\hat{x}_i|$, capped at 80):

	$i = 1$	$i = 2$	$i = 3$	$i = 4$
$\hat{x}_i$	1.0911	−1.5275	−0.2182	0.6547

LN normalizes *along the feature axis, per token*. The statistics μ, σ^2 depend only on the d numbers inside one token's vector — never on the batch or on neighbouring positions. This is the property that makes LN insensitive to sequence length and batch size.

3 The role of ε

The ε inside the square root guards against division by (near-)zero variance. If a token's features happen to be (almost) constant, $\sigma^2 \rightarrow 0$ and $1/\sqrt{\sigma^2}$ would blow up, producing huge or NaN activations. Adding ε bounds the denominator below by $\sqrt{\varepsilon}$. In our example $\sigma^2 = 1.3125$ dwarfs $\varepsilon = 10^{-5}$, so it changes $\sqrt{\sigma^2 + \varepsilon} = 1.1456483$ versus $\sqrt{\sigma^2} = 1.1456439$ — a difference in the fifth decimal, negligible here but life-saving when σ^2 is tiny.

4 Contrast with BatchNorm

BatchNorm normalizes each *feature* across the *batch* of examples; LN normalizes each *example* across its *features*. The axes are orthogonal.

	LayerNorm	BatchNorm
Normalize over	feature dim (per token)	batch dim (per feature)
Statistics depend on	one example only	whole batch
Needs running stats?	no	yes (train/eval differ)
Batch-size sensitive?	no	yes (fails at small batch)
Variable seq length?	fine	awkward (ragged batches)

For a $B \times T \times d$ batch of sequences, BatchNorm would couple a token's normalization to *which other sequences and positions* share its batch — but T varies between batches and padding pollutes the statistics. LN sidesteps all of this: a single token's output never depends on its neighbours, so a length-3 sequence and a length-3000 sequence are processed identically.

Do not confuse the *biased* variance ($\frac{1}{d} \sum$, used by LN) with the *unbiased* estimator ($\frac{1}{d-1} \sum$). LN uses the population form with divisor d , matching equation (2). Using $d-1$ here would change the example's σ^2 from $5.25/4 = 1.3125$ to $5.25/3 = 1.75$ and every downstream number with it. Also: γ and β are *learned per-feature vectors*, not scalars — they let the network undo the normalization if a feature genuinely needs a non-zero mean or non-unit scale.

5 Where this sits in the track

Link	Connection
Topic 1 (this module)	$J_{\text{LN}} = \partial \text{LN} / \partial x$ is a factor of the block Jacobian J_F .
Topic 4 (this module)	Pre- vs post-norm is about <i>where</i> this LN sits relative to the residual add.
Module 3.1	RMSNorm drops the mean-centring and β , keeping only the $1/\sqrt{\cdot}$ rescale by the root-mean-square: $\text{RMSNorm}(x) = \gamma \odot x / \sqrt{\frac{1}{d} \sum x_i^2 + \varepsilon}$. Cheaper, and standard in LLaMA-class models.

A Reproducible (NumPy)

Inputs: $x = [2.0, -1.0, 0.5, 1.5]$, $\gamma = [1, 1, 1, 1]$, $\beta = [0, 0, 0, 0]$, $\varepsilon = 10^{-5}$. Intermediates: $\mu = 0.75$, $x - \mu = [1.25, -1.75, -0.25, 0.75]$, $\sigma^2 = 1.3125$, $\sqrt{\sigma^2 + \varepsilon} = 1.1456483$. Output $\hat{x} = [1.09109, -1.52752, -0.21822, 0.65465]$.

```
import numpy as np
x = np.array([2.0, -1.0, 0.5, 1.5])
g, b, eps = np.ones(4), np.zeros(4), 1e-5
mu = x.mean() # 0.75
var = ((x - mu)**2).mean() # 1.3125 (divisor d, not d-1)
xhat = (x - mu) / np.sqrt(var + eps) # [1.09109 -1.52752 -0.21822 0.65465]
y = g * xhat + b
print(mu, var) # 0.75 1.3125
print(xhat) # [ 1.09108529 -1.52751941 -0.21821706  0.65465118]
print(xhat.mean(), xhat.var()) # ~ -2.8e-17 ~0.99999
```